1. What this small demo does

AUTOSEC asks two simple questions about text: “Does this pattern occur?” and “Does the whole command follow our grammar?” Those questions are independent. It also compares two regular rules and finds a shortest input that explains their difference.

The current app has only **Check text** and **Compare rules**. There is no dashboard, graph editor, case library, export workflow, URL panel or standalone PDA screen. Use Show steps only when a reviewer asks how an answer was obtained.

Check text starts with:

```
inspect --tag sample; (scan file)
```

Click Check input, or click Valid command. The pattern is Found because --tag matches --[a-z]+. Syntax is Valid because the flag has a value and the grouped command closes.

Click Missing value:

```
inspect --tag
```

Pattern stays Found. Syntax becomes Invalid because the flag needs a value. This is the central teaching point: finding a flag does not prove a complete command.

Click Missing bracket. The selected example is a valid command with its final) omitted. The flag still matches, but the parser rejects the unclosed group.

Compare rules uses A = --[a-z]+ and B = --[a-z][a-z]+. A needs at least one letter, B at least two. Press the compare button: --a is a shortest distinguishing input. A accepts; B rejects.

Say this: “Our project demonstrates pattern matching, nested syntax and shortest counterexamples using real formal-language algorithms behind a small interface.”

2. One input through the code

The example **inspect --tag sample; (scan file)** flows as follows:

```
App.tsx: user clicks Check input
    ↓
useComputation.run({ ...BASE, input })
    ↓
worker.ts: analyse(data.input, data.rules)
    ↓
analysis.ts: validate → signature search → lexer → parser
    ↓
worker sends the result back
    ↓
React shows Pattern and Syntax independently
```

The CheckText component keeps the typed input in React state. Submitting the form calls run(). A new browser worker receives {id, task, data}. The task is analyse; data contains the command and one fixed signature.

The worker's dispatcher calls analyse() in analysis.ts. First validateText() checks supported characters and the input bound. The signature is compiled to a minimal DFA and findMatches() searches the raw command. The literal flag occurrence is at half-open interval [8, 13): positions 8 through 12, which spell --tag.

The lexer produces these categories:

```
WORD FLAG WORD ; ( WORD WORD )
```

The parser receives the tokens, not the raw character string. grammarInfo() computes FIRST/FOLLOW and the predictive table from the productions. parseTokens() follows that table and records every action.

The returned object contains signatures, lex, parse and grammar. patternMatched is true when the signature occurs. structureAccepted is true when the complete parse accepts. React renders those separate properties; one is not inferred from the other.

Editing clears the previous result. Changing views or cancelling terminates pending work. A failed, cancelled or timed-out computation produces an incomplete error, not a new acceptance verdict.

3. How patterns become machines

A regex describes a language. --[a-z]+ means two hyphens followed by one or more lowercase letters. The editable regex engine is implemented in our TypeScript source; it is not simply a call to the browser's native regex matcher.

The actual pipeline in automata.ts is:

```
parseRegex() → syntax tree
thompson() → ε-NFA
determinize() → DFA
minimize() → minimal DFA
```

parseRegex() groups literals, classes and operators according to precedence. Thompson construction builds states and edges from that tree. An ε-edge changes state without consuming a character.

epsilonClosure() finds all states reachable using zero or more ε-edges. It includes the starting states themselves. Subset construction represents each reachable set of NFA states as one DFA state. A DFA therefore needs only one active state at a time.

Minimisation groups states that cannot be distinguished by any future input. It first removes unreachable states, then refines accepting and rejecting blocks. Equivalent states merge without changing the recognised language.

For the actual engine example ab*, the counts are 6 NFA states, 4 DFA states and 3 minimal-DFA states. It accepts a, ab and abbb, but rejects b and aa. These counts describe that expression, not every pattern.

The app does not show a machine-building laboratory anymore. The engine still performs these operations when compiling each signature or comparison rule. The paper's diagrams explain those computations.

Do not say: “We invented NFA-to-DFA conversion.” Our contribution is the small integrated experiment and explanation; the algorithms are standard.

4. How syntax and nesting work

A lexer names pieces of text: inspect is WORD, --tag is FLAG, and "sample data" is one QUOTED token. Tokenisation also preserves source positions. A parser checks the arrangement of those pieces.

Our grammar is fixed:

```
Script    → Statement Rest
Rest      → ; Statement Rest | ε
Statement → WORD Args | ( Script )
Args      → Arg Args | ε
Arg       → FLAG Value | Value
Value     → WORD | QUOTED
```

The arrow means “can be replaced by”; | means alternatives; ε means nothing. For example, Arg → FLAG Value requires a value after every flag.

FIRST tells us what can begin a production. FOLLOW tells us what may come after a nonterminal. grammarInfo() repeatedly adds symbols until these sets stop changing, then builds a table. The answers are calculated from the grammar rather than entered by hand.

parseTokens() keeps a stack and a cursor into the tokens. The top of the stack is the next thing to recognise. A nonterminal expands using the table and current token type. A terminal must match that token. Matching advances the cursor; an ε-production consumes no input.

For inspect --tag, after matching WORD and FLAG the parser still expects Value. Input has ended, so it rejects. The pattern result remains true because the raw text still contains --tag.

For a grouped statement, Statement → (Script) leaves a closing parenthesis pending while the inner Script is parsed. This stack memory explains arbitrary theoretical nesting. The standalone delimiter-PDA helper remains engine-only; it is not a separate demo feature.

Inside Show steps, the app displays tokens, parser actions and this grammar. The mathematical grammar allows arbitrary nesting; the finite implementation still has input and resource bounds.

5. Why the counterexample is shortest

Compare rules sends task = compare to worker.ts. The worker compiles A and B separately, then calls compareDfa(left.minimal, right.minimal).

A product state contains one state from A and one from B. The search starts with their two start states. For each available character, it follows that character in both machines and adds the resulting pair to a queue if unseen.

Breadth-first search processes shorter paths first. A pair is a disagreement when exactly one component accepts. The first such pair therefore provides a shortest witness. Parent links reconstruct the characters used to reach it.

For the default rules, the reconstructed characters are:

```
start → - → -- → --a
```

Before the a, neither rule accepts. After the a, A accepts and B does not. An input with fewer than three characters cannot satisfy either rule, so there is no shorter disagreement. The app shows A Accepts and B Rejects. Optional steps show the corresponding state trace.

State q4 in A and state q4 in B are labels belonging to separate machines. Identical numbers do not imply identical acceptance or meaning.

If all reachable pairs are explored with no disagreement, the languages are equivalent. Try a+ versus aa*: both mean one or more a characters.

If the start pair already disagrees, the witness is the empty string. Try a* versus a+: A accepts empty input, B does not. The interface displays ε (empty input).

The small UI always uses **whole-string comparison**. It asks whether the whole witness belongs to each language. Signature search on the other view is a separate raw-text substring search.

6. Stack, files, tests and presentation

Part	Responsibility
React	Input state, two views and displayed results.
TypeScript	Typed application and algorithm code.
Vite	Local development and production bundling.
Web Worker	Browser-thread computation and cancellation.
HTML/CSS	Plain layout, forms and optional steps.
Node / Vitest	Development tools and automated checks.

There is no backend, database, AI service or API key. The browser analyses text locally and never executes commands. Initial installation may require internet; normal use of the installed local build does not.

File	Read this to understand
src/App.tsx	CheckText, CompareRules, worker lifecycle and result display.
src/worker.ts	analyse / compile / compare task dispatch.
src/engine/analysis.ts	Signature search plus tokenisation and parsing.
src/engine/automata.ts	Regex parsing, NFA/DFA transformations and product search.
src/engine/grammar.ts	Lexer, grammar sets, table and parser.
tests/app.test.tsx	Seven current-demo interaction tests.
tests/engine.test.ts	83 engine cases and independent-language checks.

Run: npm.cmd run build, then npm.cmd run preview -- --port 4173. Open http://127.0.0.1:4173/. Keep the server running. Development uses npm.cmd run dev; tests use npm.cmd test. On other systems use npm.

Two-minute presentation: valid example → missing value → missing bracket → compare default rules → explain --a. Expand steps only if asked. Both members should be able to explain the two independent verdicts and the BFS shortest-witness argument.

90 tests and the build passed. Real browser checks covered the small demo, cancellation and two screen sizes without observed console errors. This supports readiness, not a guarantee of full marks. Rehearse on the actual presentation laptop and check the faculty rubric.